

NYC - TAXI - TO - DO

- 1) Imports. → Jan + Feb 2016
- 2) → Train → March 2016
→ Test } columns } shape of each dataframe
- 3) Visualize both Train & Test (only dataframe)
- 4) ML Problem Formulation
- 5) Performance Metrics
- 6) Data cleaning (For both train and test)
 - ↳ Univariate analysis (only on poliun)
 - 6.1) ~~Pickup latitude & longitude;~~
~~Dropoff latitude & longitude;~~
~~add to a new dataframe~~
~~after cleaning.~~
 - Don't forget to add these columns
 - 6.1.1) passenger_count
 - 6.1.2) trip_distance
 - 6.1.3) pickup_longitude
 - 6.1.4) pickup_latitude
 - 6.1.5) dropoff_longitude
 - 6.1.6) dropoff_latitude
 - ~~Univariate analysis on~~ ~~erroneous datapoints.~~
~~(Only for Train).~~

For Train & Test both

6.2) Trip Durations

6.2.1) Univariate analysis on erroneous data points (only for train)

6.2.2) Add to a new dataframe after cleaning.

↳ Convert to unix func
↳ Func. to return these new features:

- a) Trip times
- b) Pickup times
- c) Speed.

6.2.3) ~~Don't forget~~ ^{only on frame with duration} (drive)

↳ frame with durations
↳ ~~filter~~ (filtered trip times)

↳ frame with durations modified.

↳ Now do univariate analysis on modified dataframe

↳ b) Speed

↳ univariate analysis
↳ filter erroneous points

c) Trip distance ^{univariate filter}

1 & 6.2 need to be done at 1 go except
for train & Test both speed
and trip
distance

done till here

6.3) Total fare (Total amount)

↳ Univariate analysis

↳ Filter.

done till here...

6.4) Data Preparation :- (For Train & Test both)

6.4.1) Clustering/Segmentation

Jan ✓
Feb ✓
March ✓

↳ coords

↳ Func. find clusters

↳ Func find min distance

↳ Find optimum cluster size
wrt min ~~the~~ inter-cluster
distance.

Done for
Jan, Feb
March

↳ Do k-means using Minibatch
kmeans.

and predict pickup cluster
on the outlier removed
dataframe.

DO simultaneous
kmeans
using
Done for
Jan, Feb
not for
March

↳ Plotting cluster centers using
bolina after saving k-means
result of cluster centers

↳ Plotting clusters using scatter
plot.

6.4.2) Time binning :

↳ New feature pickup bin
(for Train & Test)

Done

It is made using func ~~add~~
add_pickup_bins in which
you pass the dataframe (complete)
the month & the year.
but there is no need to do
this in the modified func you
prepare again.

→ You will create 2 variables
the updated dataframe with
pickup_bins and you will
create another grouped by
dataframe - grouped by
'pickup_cluster', 'pickup_bins'

Do the complete procedure for both
Train & Test / 2016.

6.4.3) Smoothing (only for Train)

Done
after
combining
Train
& test
on

→ Func. to get unique pickup
bins using func. return avg
pickup_bins

→ Implement the above func.
→ Find each cluster number
with 0 pickups.

→ Fill in values with 0 pickups
in 2 ways:-

- i) Fill missing values with 0
- ii) Fill " " " "

avg values

Case (1): values missing at start
Case (2): " " " middle
Case (3): " " " end

↳ Do Smoothing carefully
first implement :-
i) Fill missing then Train
ii) Smoothing

Doubt in Time Series and Fourier Transform features.

7) Modelling → Baseline Models

1) Ratio of 2016 : 2015 data, i.e.,

$$R_t = \frac{P_t^{2016}}{P_t^{2015 - \text{Train}}}$$

↓
Test

2 types
of models

2) Using previous known values of 2016 data - March to predict the future values.

7.1) Model 1 — Simple Moving Averages

7.1.1) func MA_R_Predictions.

7.1.2) func MA_P_Predictions.

7.2) Model 2 — Weighted Moving Averages

7.2.1) func WA_R_Predictions

7.2.2) func WA_P_Predictions

✓ 7.3) Model 3 - Exponential Weighted Moving Averages

7.3.1) Func EA-R1 Predictions
7.3.2) Func EA-P1 Predictions

✓ [Calling all the models at Once].

✓ ↳ Comparison between baseline models.

7.4) Regression Models :-

↳ Train-Test Split

(No need as the data is already split). 70% 30% Train Test

i) Just mention size of Train & Test.

[HPT]

7.4.1) Linear Regression (Grid Search)
7.4.2) Random Forest Regressor (Random Search)
7.4.3) XgBoost Regressor (Random Search)

↳ Calculating error metric values for

various models.
→ Display Error Metrics Matrix

Time Series Features (to do later)

Exponential Smoothing

Time series forecasting method for univariate data.

It is of 3 types:-

- 1) Basic/Single Exponential Smoothing.
- 2) Double exponential smoothing
- 3) Triple exponential smoothing

● Single Exponential Smoothing (SES):

- For forecasting when there is no trend or seasonality in data.
- Single parameter (α) also called as smoothing factor/coefficient.
- It controls the rate at which the influence of observations at prior time steps decay exponentially.
- often set to a value b/w 0 and 1.
- If large value is taken, most recent past observations are paid attention to.
- If small value is taken, more history is taken into consideration.

2) Double exponential smoothing:

- Extension to exponential smoothing that explicitly adds support to trends in the univariate time series.
- In addition to α , the smoothing coefficient an additional smoothing factor is added called β which controls the decay of the influence of the change in trend.
- This supports trends that change in different ways:
 - * Additive depending whether the trend is linear.
 - * Multiplicative depending whether the trend is exponential.
- Double exponential smoothing with an additive trend is classically referred to as Holt's linear trend model.
- Dampening means reducing the size of the trend over future time steps down to a straight line which means no trend.
- Just like the trend is Additive or Multiplicative the dampening is also Additive or Multiplicative.
- In that a coefficient for dampening ϕ is used.
- Hyperparameters:
 - 1) Alpha (α) - Smoothing factor for level
 - 2) Beta (β) - " " " trend
 - 3) Trend type (Additive or Multiplicative)
 - 4) Dampen type (Additive or Multiplicative)
 - 5) Phi (ϕ) - Dampening coefficient

3) Triple exponential smoothing:

- An extension of Exponential smoothing that explicitly adds support for seasonality to the univariate time series
- Sometimes called as Holt-Winters Exponential Smoothing
- In addition to α and β a new parameter is added called γ that controls the influence on the seasonal component.
- Additionally, to ensure the seasonality is modeled correctly, the number of time steps in a seasonal period must be specified.
- Hyperparameters:
 - 1) α
 - 2) β
 - 3) γ
 - 4) Trend type
 - 5) Dampen type
 - 6) ϕ
 - 7) Seasonality type
 - 8) Period
- Most of the other theory is same as double exponential smoothing.

Fourier Transform

Return Fourier Regions - Why?

- why are we finding the fourier regions
- Fourier regions are calculated using `np.fft.fft` where in smoothed data is inserted and only 40 such values are calculated for that smoothed data.

- why are we finding fourier frequency
- Fourier frequencies are calculated using `np.fft.fftfreq` where in length of smoothed data is entered with only 40 such values.

- indices is an int data type where in ~~length~~ length of the zeroth index of fourier region + 1 is entered.

half of

- offset is a list which is calculated using the fourier frequencies, only 40 such values, for all the indices - 1 indexes.

- first part is a list made up of fourier regions for 40 such values, i.e. 1 to 40 excluding the 70th last indices.

- last part is a list made up of fourier

regions, 40 such values, ~~all~~ all in reverse order. Then again the last-part is reinitialized with all the last-part values looping through 40 from 0 to indices-1 such indexes.

- psd (power spectral densities) :-
 - list of absolute of first-part 0 to 40 indexes power of 2.
 - +
~~list~~ absolute of last-part 0 to 40 index power of 2.
- fourier frequencies is calculated as a list of fourier frequency calculated from 0 to 40, in the range of 1 to length of power spectral density's 0th index + 1th index
- np.fft.fft finds out Discrete Fourier Transform - what is this?
- np.fft.fftfreq finds out Discrete Fourier Transform Sample frequencies - what is this?

Sample Code: trip_exp_smooth.py

Grid Search Triple Exponential Smoothing

- ① def exp_smoothing_forecast(history, config)
- ② def measure_rmse(actual, predicted)
- ③ def train_test_split(data, n_test)
- ④ def walk_forward_validation(data, n_test, cfg)
- ⑤ def score_model(data, n_test, cfg, def_debug=False)
- ⑥ def grid_search(data, cfg_list, n_test, parallel=True)
- ⑦ def exp_smoothing_configs(seasonal=None)

main func.

1 / def exp_smoothing_forecast(history, config)

 t, d, s, p, b, r = config

 # history comes from train data

 history = array(history)

 model = Exponential Smoothing(history, trend=t, damped=d, seasonal=s, seasonal_periods=p)

 model_fit = model.fit(optimized=True, use_boxcox=b, remove_bias=r)

 yhat = model_fit.predict(len(history), len(history))

 return yhat[0]

Stage 7
File

Data - Smoothed_data.pickle

~~from~~
~~import~~
import

statsmodels.tsa.holtwinters
ExponentialSmoothing

Rough
idea

ExponentialSmoothing func. being
called inside exp smoothing forecast
with history and config parameters.
~~use ② to measure~~ ~~bmse~~

So use → ①

Use ② to measure RMSE.

Use ③ for train test split

Use ④ for actual calculation
in which we estimate prediction
error using rmse.

Use ⑤ for actual calling of
4th func.

Use ⑥ in which you do grid search.

Use ⑦ in which you find the
model parameters.

Run the lines inside of main
func.

For calculating Fourier Features :-

1) Frequency / Amplitude for each cluster
per each hour.

OR

2) Frequency / Amplitude for each cluster
per each day.

OR

3) Frequency / Amplitude for all the
clusters per each ~~day~~ hour

OR

4) Frequency / Amplitude for all the clusters
per each day

*

each cluster / each hour
each cluster / each day
all cluster / each hour
all cluster / each day.

Linear Regression } Not < 0.12

- HPT
- Actual training
- MAPE

Random Forest Regression } Not < 0.12

- HPT
- Actual training
- MAPE

CATBoost Examples

Weighted Moving Average features embedding
Other Moving Averages.

Research Analysis

Divide and Conquer - Based 1D CNN Human Activity Recognition using Test Data Sharpening

Authors :-

Heeryon Cho
Sang Min Yoon

1) Problem

To identify the actions performed by humans using signals collected from various sensors embedded in mobile devices.

2) Solution

Broad view: Proposal of a 1-D CNN for HAR that employs a divide and conquer-based classifier learning coupled with test data sharpening.

Detailed View: It leverages a 2-stage learning of multiple 1D CNN models;

a) They first build a binary classifier for recognizing abstract activities.

b) Then, they build 2 multi-class 1D CNN models for recognizing individual activities.

c) Then they introduce test data sharpening during prediction phase to further improve the activity recognition accuracy.

How does the paper solve the problem?

A The classifier learning is conducted via a 2 stage process:-

1) In the first stage, a binary 1D CNN model for abstract activity recognition is learned for classifying dynamic and static activities.

2) In the second stage, 2 3-class 1D CNN models are learned for classifying individual activities.

B In the prediction phase, the method first classifies dynamic and static activities using the first-stage abstract activity recognition model, and then proceeds to test data into relevant sharpening.

C After test data is sharpened, their approach inputs the sharpened test data into the relevant second-stage individual activity recognition model to output the final individual activity.

The parameters σ and α required for appropriate test data sharpening is searched and selected using the validation data after the entire 2 stage classifier learning process is completed and all 3 1D CNN models are built.

Related work (why is it not sufficient) and how this paper works better:-

They use 2 benchmark HAR datasets and show that their approach outperforms both the 1D CNN models without test data sharpening and existing state of the art approaches as they break down the multi-class problem into simpler problem units and introduce test data sharpening in the prediction phase. By this they can achieve better HAR performance.

Key contributions that the paper claim:

The contributions of this paper are 2-fold:

- 1) They propose a divide and conquer approach for building 2-stack HAR that incorporates test data sharpening during prediction phase to enhance HAR performance.
- 2) They present a systematic method for identifying useful parameters, i.e., σ and α needed for test data sharpening.

I Related Work details: Existing Neural Network

- MLP has shown competitive activity recognition performance, the algorithm is known to output poor recognition performance when it falls into local optima.
- Moreover, adding many hidden layers to MLP was difficult due to vanishing gradient problem during back propagation.
- To overcome such limitations of shallow classifiers, deep neural networks learning methods were introduced.
- Some implemented deep activity recognition models based on deep belief networks (DBN).
- The first layer of their DBN consisted of Gaussian-binary Restricted Boltzmann Machine (~~RBM~~^{RBM}), which modeled the energy content in continuous accelerometer data; the subsequent layers were modeled using binary-binary RBMs.
- CNN-based algorithms have been applied to HAR for its advantages in capturing local dependency of activity signals and preserving feature scale invariance.

- Other approaches exploited deep Recurrent Neural Network (RNN) or combined Long Short-Term Memory (LSTM) RNN with CNN.
- Ordóñez and Reggen proposed DeepConvLSTM that combined convolutional and recurrent layers.
- Here, abstract representations of input sensor data were extracted as feature maps in the convolution layer, and the temporal dynamic of feature maps & activations were modelled in the recurrent layers.
- Edel & Koppe proposed binarized, bidirectional LSTM-RNNs that reduced memory consumption and replaced most of the arithmetic operations with bitwise operations, achieving an increase in power-efficiency.

II Related Work Details: 2-stage classifier learning techniques

- Read from Research paper.
- Only related work is mentioned.

III Feature Processing

- Read from Research paper

III Actual Models

1) Model 1 — Part I

① Training data

(window size = 2) 156

② Conv 1 (155), 30 filters

③ Conv 2 (154), 50 filters

④ Conv 3 (153), 100 filters

⑤ Conv 4 (152), 300 filters

⑥ Conv 5 (151), 500 filters

⑦ Flatten ⑧ Dense/Dropout ⑨ Softmax layer.

2) Model 1 - Part 2

1) Input (156) window size = 3.
↓

2) Conv 1, (154) 100 filters
↓

3) Maxpooling (152), 100 filters
↓

4) Conv 2 (150), 500 filters

5) Flatten

6) Dense/ Dropout

7) Softmax

Deep Learning Assignment: LSTM on Donors

Choose

1. Dataset

Zipped folder

miscellaneous files

ipynb

train.csv

Features/Columns

1. project_id
2. project_title
3. project_grade_category
4. project_subject_categories
5. school_state
6. project_subject_subcategories
7. project_resource_summary
8. project_essay_1
9. " " " 2
10. " " " 3
11. " " " 4
12. project_submitted_datetime
13. teacher_id
14. teacher_prefix
15. teacher_number - of previously posted projects
16. teacher_id
17. teacher_prefix

resources.csv

1. id
2. description
3. quantity
4. price

class label

project is approved

Notes on Essay data

project-essay-1 } After May 2016
project-essay-2 }

2: Data Analysis

Shape of train (109248, 17)

Shape of resources (1541272, 4)

→ average

{ 85% projects are accepted.
15% " " not " }

Q 2 additional activities:-

- 1) Univariate analysis
- 2) Design a new column for a new feature.

3. Text preprocessing

- Essay 1+2+3+4
- Title

decontracted
preprocessing of essays
" " Title

4. Preparing Data Matrix

Categorical

1. School State
2. Clean categories
3. Sub
4. Project grade category
5. Teacher prefix

Text data

1. Project title
2. ~~at~~ Text Essay
Project Resource Summary

BOW
TFIDF,
AvgW2V,
Avg TFIDF

Numerical

1. Quantity
2. Teacher - no. of - previously posted projects
3. price.

BOW
cannot
Vectorized

min df = 10 means we use words
which appear in 10 docs.

TR1DF

W2V — TFIDFW2V
↓ Aug W2V

glove_vectors → Aug W2V

5. Introduction to Functional API Tensor flow